

CASE STUDY

Enterprise AI Safety & Transparency Platform

Building OWASP LLM Top-10 compliant infrastructure for enterprise AI deployment

- ~95% complete — core security pipeline, access control, and audit layer implemented



01 Overview

The Enterprise AI Safety & Transparency Platform is a security and compliance layer purpose-built for organizations deploying LLMs in production. It treats prompt injection, data leakage, and unaudited model behavior as first-class security concerns — the same way a traditional platform treats SQL injection or broken authentication. The system currently sits at roughly 95% completion: the security pipeline, access control, monitoring, and audit-logging layers are implemented and map to all 10 OWASP LLM Top-10 categories. Remaining work is concentrated in expanding automated test coverage and a formal third-party penetration test.

02 The Problem

Most teams adopting LLMs inherit their existing web-application security posture and assume it's sufficient. It isn't. An LLM endpoint introduces attack surface that doesn't map cleanly onto the traditional OWASP Top 10: a user can manipulate model behavior through the prompt itself, models can leak sensitive information surfaced through context, and — critically for regulated industries — there's often no audit trail explaining why a model produced a given output. Meanwhile, compliance frameworks like GDPR, CCPA, SOC 2, and ISO 27001 were written before generative AI existed, leaving enterprises to interpret how requirements like data minimization apply to a system that generates novel text on demand. This platform exists to close that gap.

03 Design Principles

- Defense in depth — no single control is trusted to catch everything; each layer assumes the one before it can fail.
- Everything is audited — every request, decision, and moderation action is logged with enough context to reconstruct what happened and why.
- Compliance is a design input, not a retrofit — GDPR, SOC 2, and ISO 27001 requirements shaped the data layer from the start.

04 Architecture Deep-Dive

How the 8 pipeline stages (see the project overview) group into four defensive layers.

01 Perimeter & Identity

STAGES 1-2 • API GATEWAY, AUTHENTICATION

Every request passes through two checkpoints before it touches application logic. The gateway handles rate limiting and TLS 1.3 termination — cheap, stateless checks that stop the bulk of automated abuse before it costs any compute. Authentication then closes the single most common enterprise breach vector: a leaked credential. JWT sessions are paired with TOTP-based 2FA, and API keys are scoped and rotated independently of user sessions, so a compromised key doesn't imply a compromised account.

02 Content Safety Pipeline

STAGES 3, 5 • SECURITY PIPELINE, ETHICS CHECKER

This is the layer that doesn't exist in a traditional web stack, because the attack surface it defends against didn't exist before LLMs. A BERT-based classifier screens every incoming prompt for injection patterns before it reaches the model, while PII masking strips sensitive fields out of both the input and the eventual context window. On the output side, generated content runs through OpenAI's Moderation API plus custom classifiers tuned to the platform's specific risk areas — bias and policy-violating content get caught before a response reaches a user, not after a complaint.

03 Authorization & Resilience

STAGES 4, 6 • ACCESS CONTROL, REQUEST PROCESSING

Role-based access control is enforced at the ORM layer rather than the API layer, which matters more than it sounds — a bug in a route handler can accidentally skip an API-level check, but it can't bypass a query that's already scoped by role. Downstream, request processing runs through an async queue with exponential backoff, so a slow or failing dependency degrades response time instead of taking the whole request down with it.

04 Observability & Accountability

STAGES 7-8 • MONITORING & ALERTS, DATA LAYER

Real-time anomaly detection watches the full request stream continuously, not on a polling schedule — the goal is catching an attack pattern while it's still forming. Underneath that, the data layer is where compliance claims either become real or stay theoretical: PII is encrypted at rest, every action writes to an immutable audit trail, and retention policies are explicit rather than implied. It's also why PostgreSQL is paired with TimescaleDB specifically — a general-purpose relational database struggles with the time-series query patterns an actual audit ("show every access to this record in the last 90 days") requires.

05 **Production & Compliance**

Deployment runs on Docker and Kubernetes across Azure and AWS, with PostgreSQL plus TimescaleDB handling audit-log data specifically — a general-purpose relational database alone struggles with the query patterns compliance audits actually need, like surfacing every access to a given user's data in a fixed window. Redis handles rate-limiting and session state; Celery processes async jobs. The result maps onto all 10 OWASP LLM Top-10 categories, with the data layer structured to support GDPR, CCPA, SOC 2, and ISO 27001 audit requirements.

Deployment

Docker + Kubernetes on Azure/AWS

Database

PostgreSQL + TimescaleDB for audit logs

Cache

Redis for rate limiting + sessions

Queue

Celery for async jobs

Monitoring

ELK stack + custom dashboards

Compliance

GDPR/CCPA/SOC 2/ISO 27001 ready

06 Results

As it stands: 100+ API endpoints across 8 routers, full OWASP LLM Top-10 coverage, JWT + 2FA TOTP authentication, and zero known security vulnerabilities in the current implementation. Monitoring runs continuously rather than on a scheduled basis, and every action — authentication, moderation decisions, data access — is captured in the audit trail.

- 100+ API endpoints across 8 routers
- 10/10 OWASP LLM Top-10 compliance
- JWT + 2FA TOTP authentication
- 0 known security vulnerabilities
- Real-time threat detection
- 24/7 automated monitoring
- Complete audit trail logging

07 What's Next

The remaining work is concentrated in two places: expanding automated test coverage around the anomaly-detection thresholds, which are currently tuned manually against synthetic attack patterns, and a formal third-party penetration test before treating any compliance claim as audit-ready rather than architecturally-ready.

08 Tech Stack

FastAPI

PostgreSQL

Redis

Docker

Kubernetes

Celery

PyJWT

OpenAI Moderation

ELK Stack

Prometheus

Grafana